

Clarke Error Grid Analysis

Professional implementation of Clarke's Error Grid Analysis for evaluating the clinical accuracy of blood glucose predictions.

Overview

The Clarke Error Grid Analysis (EGA) is a widely accepted method for assessing the clinical significance of differences between blood glucose measurements. It was introduced by William L. Clarke in 1987 and has become the gold standard for evaluating glucose monitoring systems.

Clinical Zones

The Clarke EGA divides the prediction space into five zones based on clinical impact:

Zone A: Clinically Accurate

- **Definition:** Values that differ from reference by $\leq 20\%$ or both values are < 70 mg/dL
- **Clinical Impact:** No clinical impact; accurate measurements
- **Target:** $> 95\%$ of predictions
- **Treatment Decision:** Correct

Zone B: Benign Errors

- **Definition:** Values that differ from reference but lead to benign or no treatment
- **Clinical Impact:** Minimal clinical impact
- **Acceptable:** $< 5\%$ of predictions
- **Treatment Decision:** Acceptable deviation

Zone C: Overcorrection Errors

- **Definition:** Values leading to unnecessary corrective treatment
- **Clinical Impact:** Overcorrection of acceptable glucose levels
- **Target:** 0% of predictions
- **Treatment Decision:** Potentially harmful overcorrection

Zone D: Dangerous Failure to Detect

- **Definition:** Values that fail to detect hypoglycemia or hyperglycemia
- **Clinical Impact:** Dangerous; failure to treat when necessary
- **Target:** 0% of predictions
- **Treatment Decision:** Dangerous failure

Zone E: Erroneous Treatment

- **Definition:** Values that would lead to opposite treatment direction
- **Clinical Impact:** Highly dangerous; wrong treatment

- **Target:** 0% of predictions
- **Treatment Decision:** Critically dangerous

Usage

Basic Analysis

```
from benchmark.evaluation.clarke_ega import ClarkeEGA
import numpy as np

# Your data
y_true = np.array([120, 85, 150, 200, 75]) # Reference glucose (mg/dL)
y_pred = np.array([118, 88, 145, 195, 78]) # Predicted glucose (mg/dL)

# Create analyzer
clarke = ClarkeEGA()

# Perform analysis
results = clarke.analyze(y_true, y_pred)

# View results
print("Clarke Error Grid Analysis Results:")
print(f"Total points: {results['total_points']}")
print("\nZone Distribution:")
for zone in ['A', 'B', 'C', 'D', 'E']:
    count = results['zone_counts'][zone]
    percentage = results['percentages'][zone]
    print(f"Zone {zone}: {count} points ({percentage:.2f}%")
```

Visualization

```
import matplotlib.pyplot as plt

# Create figure and axis
fig, ax = plt.subplots(figsize=(10, 10))

# Plot on existing axes
clarke.plot_on_axes(
    ax=ax,
    y_true=y_true,
    y_pred=y_pred,
    point_size=40,
    color_zones=True,
    color_points=True
)

plt.tight_layout()
plt.savefig('clarke_ega.png', dpi=300)
plt.show()
```

Generate Complete Figure

```
# Generate standalone figure
fig = clarke.plot(
    y_true=y_true,
    y_pred=y_pred,
    title="Clarke Error Grid Analysis",
    save_figure=True,
    filename='clarke_analysis.png'
)
```

Methods

`analyze(y_true, y_pred)`

Performs Clarke EGA on prediction data.

Parameters:

- `y_true` (array-like): Reference glucose values in mg/dL
- `y_pred` (array-like): Predicted glucose values in mg/dL

Returns:

- `dict`: Results dictionary containing:
 - `zones` (ndarray): Zone classification (1-5) for each point
 - `zone_counts` (dict): Count of points in each zone (A-E)
 - `percentages` (dict): Percentage of points in each zone (A-E)
 - `total_points` (int): Total number of points analyzed

Example:

```
results = clarke.analyze(y_true, y_pred)
print(f"Zone A: {results['percentages']['A']:.1f}%")
```

`plot(y_true, y_pred, **kwargs)`

Creates a complete Clarke EGA plot.

Parameters:

- `y_true` (array-like): Reference glucose values
- `y_pred` (array-like): Predicted glucose values
- `figsize` (tuple): Figure size in inches (default: (8, 8))
- `point_size` (float): Size of data points (default: 30)

- **title** (str): Plot title (default: "Clarke Error Grid Analysis")
- **save_figure** (bool): Whether to save figure (default: False)
- **filename** (str): Filename for saved figure (default: 'Clarke_EGA')
- **alpha** (float): Zone fill transparency (default: 0.5)
- **color_zones** (bool): Whether to color zones (default: True)
- **color_points** (bool): Whether to color points by zone (default: True)

Returns:

- **matplotlib.figure.Figure**: The generated figure

Example:

```
fig = clarke.plot(
    y_true=y_true,
    y_pred=y_pred,
    title="My Model - Clarke EGA",
    save_figure=True,
    filename='my_clarke_analysis.png',
    point_size=50
)
```

plot_on_axes(ax, y_true, y_pred, **kwargs)

Plots Clarke EGA on an existing matplotlib axes.

Parameters:

- **ax** (matplotlib.axes.Axes): The axes to plot on
- **y_true** (array-like): Reference glucose values
- **y_pred** (array-like): Predicted glucose values
- **point_size** (float): Size of data points (default: 30)
- **alpha** (float): Zone fill transparency (default: 0.5)
- **color_zones** (bool): Whether to color zones (default: True)
- **color_points** (bool): Whether to color points by zone (default: True)

Returns:

- **matplotlib.axes.Axes**: The modified axes

Example:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))

# Plot Clarke EGA on first axes
clarke.plot_on_axes(ax1, y_true, y_pred)

# Plot something else on second axes
```

```
ax2.plot(y_true, y_pred, 'o')
ax2.set_title("Scatter Plot")

plt.tight_layout()
plt.show()
```

Zone Classification Algorithm

The Clarke EGA uses a piecewise linear approach to classify each (reference, prediction) pair:

Mathematical Boundaries

Zone A (Clinically Accurate):

- Both reference and prediction ≤ 70 mg/dL, OR
- Prediction within 20% of reference

Zone B (Benign Errors):

- Reference ≥ 180 mg/dL and prediction ≤ 70 mg/dL, OR
- Reference ≤ 70 mg/dL and prediction ≥ 180 mg/dL, OR
- Prediction 20-30% from reference

Zone C (Overcorrection):

- Reference > 70 and < 290 mg/dL
- Prediction $>$ reference +110 mg/dL, OR
- Prediction $<$ reference -110 mg/dL

Zone D (Dangerous Failure):

- Reference ≥ 240 mg/dL and prediction ≤ 70 mg/dL, OR
- Reference ≤ 70 mg/dL and prediction ≥ 180 mg/dL

Zone E (Erroneous Treatment):

- Reference ≤ 70 and prediction ≥ 180 mg/dL, OR
- Reference ≥ 180 and prediction ≤ 70 mg/dL

The implementation uses optimized NumPy operations for efficient batch processing.

Clinical Interpretation

Regulatory Standards

FDA Requirements (CGM Systems):

- 99% of values in Zones A+B
- 95% in Zone A

- 0% in Zones D+E

Clinical Research:

- Zone A: Excellent clinical accuracy
- Zone A+B $\geq 95\%$: Acceptable for clinical use
- Any values in Zone E: System needs improvement

Model Quality Assessment

Zone A Percentage	Clinical Assessment	Recommendation
>95%	Excellent	Ready for clinical deployment
90-95%	Good	Acceptable with monitoring
85-90%	Moderate	Needs improvement
<85%	Poor	Not recommended for clinical use

Example Interpretation

```

results = clarke.analyze(y_true, y_pred)

zone_a = results['percentages']['A']
zone_b = results['percentages']['B']
zone_de = results['percentages']['D'] + results['percentages']['E']

print("Clinical Assessment:")
if zone_a > 95 and zone_de == 0:
    print("✓ Excellent: Ready for clinical deployment")
elif zone_a > 90 and zone_de < 1:
    print("✓ Good: Acceptable for clinical use")
elif zone_a > 85:
    print("⚠ Moderate: Needs improvement")
else:
    print("✗ Poor: Not recommended for clinical use")

print(f"\nKey Metrics:")
print(f"  Zone A: {zone_a:.1f}%")
print(f"  Zone A+B: {zone_a + zone_b:.1f}%")
print(f"  Zone D+E: {zone_de:.1f}%")

```

Visualization Features

Color Schemes

The implementation uses intuitive color coding:

- **Zone A:** Green (clinically accurate)

- **Zone B:** Yellow-green (benign errors)
- **Zone C:** Yellow (overcorrection)
- **Zone D:** Orange (dangerous failure)
- **Zone E:** Red (erroneous treatment)

Plot Elements

1. **Zone Boundaries:** Black lines showing zone divisions
2. **Perfect Agreement Line:** Dashed diagonal (45°) line
3. **Data Points:** Colored by zone classification
4. **Legend:** Zone descriptions with counts and percentages
5. **Grid:** Light gridlines for easy reading
6. **Axes:** Labeled with glucose concentration (mg/dL)

Customization

```
# Custom styling
fig = clarke.plot(
    y_true=y_true,
    y_pred=y_pred,
    figsize=(12, 12), # Larger figure
    point_size=60,     # Larger points
    alpha=0.3,         # More transparent zones
    color_zones=True,  # Show zone colors
    color_points=True  # Color points by zone
)

# Customize further
ax = fig.axes[0]
ax.set_title("My Custom Title", fontsize=16, fontweight='bold')
ax.grid(True, alpha=0.5, linestyle='--')
plt.tight_layout()
plt.savefig('custom_clarke.png', dpi=300, bbox_inches='tight')
```

Advanced Usage

Batch Processing

```
# Process multiple models
models = {
    'LSTM': (y_true_lstm, y_pred_lstm),
    'GRU': (y_true_gru, y_pred_gru),
    'Transformer': (y_true_trans, y_pred_trans)
}

clarke = ClarkeEGA()
results_summary = {}
```

```

for model_name, (y_true, y_pred) in models.items():
    results = clarke.analyze(y_true, y_pred)
    results_summary[model_name] = results['percentages']

    print(f"\n{model_name}:")
    print(f"    Zone A: {results['percentages']['A']:.2f}%")
    print(f"    Zone B: {results['percentages']['B']:.2f}%")

# Compare models
best_model = max(results_summary.keys(),
                  key=lambda m: results_summary[m]['A'])
print(f"\nBest Model: {best_model}")

```

Integration with Other Metrics

```

from benchmark.evaluation.metrics import BGMetrics

# Calculate both statistical and clinical metrics
metrics = BGMetrics.calculate_comprehensive_metrics(y_true, y_pred)

print("Comprehensive Evaluation:")
print(f"    RMSE: {metrics['rmse']:.2f} mg/dL")
print(f"    MAE: {metrics['mae']:.2f} mg/dL")
print(f"    Clarke Zone A: {metrics['clarke_zones']['A']:.1f}%")
print(f"    Clarke Zone A+B: {metrics['clarke_zones']['A'] +
metrics['clarke_zones']['B']:.1f}%")

# Clinical assessment
if metrics['rmse'] < 15 and metrics['clarke_zones']['A'] > 95:
    print("✓ Model meets clinical standards")
else:
    print("⚠ Model needs improvement")

```

Time-Series Analysis

```

import pandas as pd

# Analyze performance over time
df = pd.DataFrame({
    'timestamp': timestamps,
    'y_true': y_true,
    'y_pred': y_pred
})

# Sliding window analysis
window_size = 100
results_over_time = []

```

```

for i in range(0, len(df) - window_size, 10):
    window = df.iloc[i:i+window_size]
    results = clarke.analyze(window['y_true'].values,
                             window['y_pred'].values)
    results_over_time.append({
        'timestamp': window['timestamp'].iloc[0],
        'zone_a': results['percentages']['A']
    })

# Plot Zone A percentage over time
df_results = pd.DataFrame(results_over_time)
plt.figure(figsize=(12, 6))
plt.plot(df_results['timestamp'], df_results['zone_a'])
plt.axhline(y=95, color='r', linestyle='--', label='Clinical threshold')
plt.ylabel('Zone A (%)')
plt.xlabel('Time')
plt.title('Clarke Zone A Performance Over Time')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('zone_a_over_time.png')

```

References

1. Original Paper:

Clarke, W. L., Cox, D., Gonder-Frederick, L. A., Carter, W., & Pohl, S. L. (1987). Evaluating clinical accuracy of systems for self-monitoring of blood glucose. *Diabetes Care*, 10(5), 622-628.

2. FDA Guidance:

U.S. Food and Drug Administration. (2020). "Self-Monitoring Blood Glucose Test Systems for Over-the-Counter Use."

3. ISO Standards:

ISO 15197:2013. "In vitro diagnostic test systems — Requirements for blood-glucose monitoring systems for self-testing in managing diabetes mellitus."

4. Clinical Applications:

- Kovatchev, B. P., et al. (2004). "Evaluating the accuracy of continuous glucose-monitoring sensors." *Diabetes Care*, 27(8), 1922-1928.

Acknowledgements

This project adapts code from: Edgar Guevara (2025). Clarke Error Grid Analysis. MATLAB Central File Exchange. <https://www.mathworks.com/matlabcentral/fileexchange/20545-clarke-error-grid-analysis> (Retrieved October 7, 2025).

Licensed under BSD 2-Clause. The original license text is included in [License](#)

Modifications by <Beatriz Fulgencio/ Puc Minas> (2025): refactored to Python, added coloured zones and labels.

See Also

- [Parkes Error Grid Analysis](#) - Consensus error grid for Type 1 diabetes
- [Evaluation Module README](#) - Complete evaluation module documentation
- [Metrics Documentation](#) - Statistical metrics implementation

License

This implementation is part of the BG-Forecasting project. See the main LICENSE file for details.